

[← Back to Blog](#)

A good AGENTS.md is a model upgrade. A bad one is worse than no docs at all.

Apr 22, 2026

Last updated: Apr 23, 2026



Slava Zhenylenko



We pulled dozens of **AGENTS.md** files from across our monorepo and measured their effect on code generation. The best ones gave our coding agent a quality jump equivalent to upgrading from Haiku to Opus. The worst ones made the output worse than having no **AGENTS.md** at all.

That gap was surprising enough that we built a systematic study around it.

What we found: most of what people put in **AGENTS.md** either doesn't help or actively hurts, and the patterns that work are specific and learnable.

The same file can help one task and hurt another by 30%

A single **AGENTS.md** isn't uniformly good or bad. The same file boosted **best_practices** by 25% on a routine bug fix and dropped **completeness** by 30% on a complex feature task in the same module.

On the bug fix, a decision table for choosing between two similar data-fetching approaches helped the agent pick the right pattern immediately and stay within codebase standards. On the feature task, the agent read that same file, got pulled into the reference section, opened dozens of other markdown files trying to verify its approach against every guideline, created unnecessary abstractions, and shipped an incomplete solution.

Different blocks of the document had opposite effects on different tasks.

What follows is which patterns work, which fail, and how to tell which is which for your codebase.

How we measured this

We used AuggieBench, one of our internal eval suites, to evaluate how well agents do our internal dev work. We start with high-quality PRs from a large repo that reflect typical day-to-day agent tasks, set up the environment and prompt, and ask the agent to do the same task. Then we compare its output against the golden PR, the version that actually landed after review by multiple senior engineers. We filtered out PRs with scope creep or known bugs

then push details into reference files the agent can load on demand. Keep each reference's scope clear so the agent knows when to pull it in.

The 100–150 line **AGENTS.md** files with a handful of focused reference documents were the top performers in our study, delivering 10–15% improvements across all metrics in mid-size modules of around 100 core files. Once the main file got longer than that, the gains started reversing.

2. Procedural workflows take agents from failing to finishing

Describing a task as a numbered, multi-step workflow was one of the strongest patterns we measured. A well-written workflow can move the agent from unable to complete a task to producing a correct solution on the first try.

One example from our codebase: a six-step workflow for deploying a new integration. The agent followed it step by step. The share of PRs with missing wiring files dropped from 40% to 10%, and the agent finished faster on average. **Correctness** went up 25%. **Completeness** went up 20%.

For complex workflows, keep the main file concise and use reference files for branching cases.

3. Decision tables resolve ambiguity before the agent writes code

When your codebase has two or three reasonable ways to do something, decision tables force the choice up front. This is the pattern that most directly improved adherence to

forces the choice up front. This is the pattern that most directly improved adherence to codebase conventions.

Example: resolving React Query vs Zustand for state management.

Question	→ React Query	→ Zustand
Server is the only data source?	✓	
Multiple code paths mutate this state?		✓
Need optimistic updates mixed with local state?		✓

PRs in this area scored 25% higher on **best_practices**. The table resolved the ambiguity before the agent wrote a single line of code.

4. Examples from the real codebase improve code reuse

Short snippets of 3–10 lines from actual production code improved reuse and pattern adherence. Keep it to a few examples that are most relevant and not duplicative. More than that and the agent starts pattern-matching on the wrong thing.

Example: we included copy-paste templates for Redux Toolkit primitives: **createSlice** with typed initial state, **createAsyncThunk** with proper error handling, and the typed **useAppSelector** hook.

code_reuse went up 20%. The agent followed the template instead of inventing its own state management pattern, and the codebase stayed consistent.

5. Domain-specific rules still matter

This is the pattern most people already associate with **AGENTS.md**: language- or org-specific gotchas.

Example: Use **Decimal** instead of **float** for all financial calculations. The agent

Example: Use `Decimal` instead of `float` for all financial calculations. The agent catches truncation, rounding, and precision issues that it would otherwise miss. **best_practices** improves whenever the rule is directly relevant to the task.

This works when the rule is specific and enforceable. It stops working when you stack dozens of them. See the overexploration section below.

6. Pair every "don't" with a "do"

Warning-only documentation consistently underperformed documentation that paired prohibitions with a concrete alternative.

If you add **Don't instantiate HTTP clients directly**, pair it with **Use the shared `ApiClient` from `lib/http` with the `retry` middleware.**

The first on its own makes the agent cautious and exploratory. The pair tells it what to do and moves on.

AGENTS.md files with 15+ sequential "don'ts" and no "dos" caused the agent to over-explore, stay conservative, and do less work. More on that below.

7. Keep your code modular, and **AGENTS.md** too

The best-performing agent docs described relatively isolated submodules. Mid-size modules, around 100 core files, with a 100–150 line **AGENTS.md** and a few reference documents, were where we saw the 10–15% cross-metric gains. Examples: UI components of the client, standalone services.

Huge, cross-cutting **AGENTS.md** files at the repo root underperformed module-level ones. But the document itself is only part of the story.

In our study, the worst-performing **AGENTS.md** files were the ones sitting on top of massive surrounding documentation. One module had 37 related docs totaling about 500K characters. Another had 226 docs totaling over 2MB. In both cases, removing just the **AGENTS.md** barely changed agent behavior. The agent kept finding and reading the surrounding doc sprawl, and the sprawl was the problem.

If your **AGENTS.md** is good but your module has 500K of specs around it, the specs are what the agent is reading. Fix the documentation environment, not just the entry point.

Where AGENTS.md falls short

The overexploration trap

This is the most common failure mode we observed, and it's essentially context rot.

Two patterns cause it:

1. Too much architecture overview.

The agent gets pulled into reading documentation files, sometimes dozens of them, trying to "better understand the architecture." It loads tens or hundreds of thousands of tokens of context, and the output gets worse.

Example: an AGENTS.md included a full service topology covering the event bus, message queues, API gateway routing, and shared middleware layers, with reasoning for every architectural decision. The task: a two-line config change. The agent read 12 documentation files trying to understand the architecture before touching code, loaded about 80K tokens of irrelevant context, got confused about which service owned the config, and produced an incomplete fix. **completeness** dropped 25%.

Fix: keep architecture descriptions concise and isolated. Vague descriptions of component responsibilities push the agent into exploration mode. Highlight boundaries. Focus on the *what*, not the *why*.

2. Excessive warnings

A big section of "don'ts" without matching "dos" produces a specific failure. The agent reads each instruction, tries to figure out whether it applies to the current task, and starts verifying its solution against every single warning. With 30–50 warnings, that means reading migration scripts, checking API version compatibility, and exploring auth middleware code, even on a

scripts, checking API version compatibility, and exploring auth middleware code, even on a task where none of it matters.

Example: an AGENTS.md with 30+ "don't" rules covering database migrations, API versioning, deployment safety, and auth boundaries. The task: a simple CRUD endpoint. The agent checked each warning for relevance and explored code it didn't need to touch. The PR took twice as long and was 20% less complete on average.

Fix: keep the core gotchas in the main file and move the majority into reference files. Pair every "don't" with a "do" whenever possible.

New patterns break old documentation

If you're introducing a pattern that doesn't exist in your codebase yet, **AGENTS.md** can actively steer the agent in the wrong direction.

Example: the AGENTS.md documented existing REST + polling patterns. The task was to build real-time collaborative editing using WebSockets. The agent followed the docs and built a polling-based solution, technically functional but architecturally wrong. The golden PR used WebSockets with a completely different data flow.

The **fix** isn't a better AGENTS.md. It's spec-driven development for net-new architecture.

Know what you're optimizing for

Different patterns move different metrics. Pick the patterns that target the problem you actually have.

If you want to improve...

Use this pattern

Reuse of existing code

Several clear and relevant examples from the prod code

Following established practices in the codebase	Decision tables for components and libraries
Ensuring proper wiring of big features	Procedural AGENTS.md
Handling of gotchas	"Don't" paired with "Do"
Context rot	Progressive disclosure of information via reference files
Context rot	Clear logical separation of what is in different reference files. Outline In AGENTS.md what exactly is there, but go no deeper
Context rot	Obvious advice, but AGENTS.md should only contain guidance relevant to the surrounding code

How agents actually find your docs

Before deciding how to migrate your existing documentation, it helps to know what the agent actually reads. We traced documentation discovery across hundreds of sessions. The discovery rates are lopsided enough to shape migration priorities.

- **AGENTS.md** files are discovered automatically in 100% of cases, for every file in the hierarchy from the working directory by most harnesses.
- References out of **AGENTS.md** are loaded on demand and read in over 90% of sessions when the agent has a reason to pull them in.
- Directory-level **README.md** files aren't auto-loaded, but the agent reads them in 80%+ of sessions when it's working in that directory.

After that, discovery falls off a cliff.

- Nested **READMEs**, meaning **README** files in subdirectories the agent isn't currently working in, get discovered only about 40% of the time.
- Orphan docs in **_docs/** folders that nothing references get read in under 10% of sessions. One service in our codebase had 30K of detailed protocol design, throttling rules, and security docs in **_docs/**. The agent never opened most of them across dozens of sessions.

AGENTS.md is the only documentation location with reliable discovery. If something needs to be seen, it either lives there or is directly referenced from there. Moving the content into a referenced location is usually higher leverage than writing more docs.

Migrating existing docs

Every company already has READMEs, architecture docs, and design specs scattered across the repo. Here's how to turn that into something an agent can actually use.

Should you just rename your README.md to AGENTS.md?

README.md and **AGENTS.md** serve different audiences, but they can be reused. Agents are good enough at codebase summarization now that human-oriented docs are less necessary than they used to be. You can either write an agentic doc from scratch, or reuse your **README.md**. If you reuse it, trim it aggressively. Keep it short, follow the patterns above, and cut any section that's there for humans to skim.

When to keep existing documentation

If the docs are high quality, current, to the point, and have examples, reuse them. Reference them from module- or folder-level **AGENTS.md** files. Don't put more than 10–15 references in a single **AGENTS.md** and keep the context lean. And audit the surrounding environment: if the module around your **AGENTS.md** has dozens of architecture docs and spec files, the agent will find and read them whether you reference them or not. A focused 150-line **AGENTS.md** sitting on top of 500K of surrounding specs won't save the agent from the specs.

AGENTS.md isn't the only path

Agents find reference material through grep and semantic search too. About half of all search-result hits in our traces came from those tools, not from **AGENTS.md** references. If you're keeping legacy documentation, make sure the docs include relevant code examples and descriptive text that's searchable. A well-structured **AGENTS.md** gives you more control over what ends up in the context window, but it isn't the only way in.

What this study didn't cover

We focused on one-shot trajectories and the agent's ability to finish coding tasks without human intervention. We didn't look at best practices for maintaining **AGENTS.md** over time, though we're exploring that now. We also didn't cover operational, interactive, or analytics tasks. Those are coming in future posts.

WRITTEN BY

Slava Zhenylenko

Member of Technical Staff

Slava is an applied AI engineer with over a decade of experience and worked across wide range of domains, such as GenAI, CV, Deep Learning and the operationalization of AI systems.

GET STARTED

Give your codebase the agents it deserves

Install Augment to get started. Works with codebases of any size, from side projects to enterprise monorepos.

Install Augment →

Contact Sales

PRODUCT

- Agent
- Intent
- Code Review
- Slack
- CLI
- Pricing

RESOURCES

- Customers
- Docs
- Blog
- Guides
- Learn
- Tools
- AI Engineering Playbook

COMPANY

- Careers
- Press Inquiries
- Press Kit
- Contact Sales
- Contact Support
- Changelog
- Privacy & Security
- Trust Center
- Status Page

LEGAL

- Cookie Policy
- Privacy Policy
- SLA and Support Policy
- Terms of Service